BLUE TEAM TECHNICAL DOCUMENTATION • SOC PORTFOLIO

Network Forensics & Incident Analysis Report

Hack The Box Sherlock Investigation: **PCAP Analysis**

1. Document Control & Metadata

Field	Details
Author & Analyst	Jonathan Di Genova (Junior SOC Analyst & Cybersecurity Specialist)
Date	November 13, 2026
Platform & Lab	Hack The Box Sherlocks — *PCAP Analysis* (Category: SOC, Difficulty: Very Easy)
Primary Artifact	`Pcap_Analysis.pcapng`
Tooling Stack	`tshark`, `awk`, Kali Linux CLI
Classification	Portfolio & Technical Study Document (Spoiler-Free Compliance)

2. Case Overview & Scope

Network traffic captured from a workstation assigned to user **P13** was submitted for Blue Team forensic triage following indicators of an unauthorized or suspicious outbound file transfer. The primary objective of this investigation was to reconstruct the network communication flow, identify the communicating endpoints, inspect the application-layer upload mechanism, fingerprint the target web infrastructure, and measure the exact network transmission duration of an encrypted payload.

In adherence to Hack The Box platform guidelines, this document focuses strictly on investigative methodology, protocol behavior, and defensive takeaways without disclosing raw task answer strings.

3. Personal Investigation Timeline

1. **Phase 1 — Traffic Triage & Stream Isolation:** Parsed `Pcap_Analysis.pcapng` using `tshark` display filters (`http.file_data`) to isolate HTTP conversations carrying file payloads from background network chatter, pinpointing the specific TCP stream responsible for the upload.
2. **Phase 2 — Endpoint & Server Enumeration:** Extracted source and destination IPv4 pairs across general communications and the isolated upload session. Inspected HTTP headers to identify the backend web server software and the server-side upload handler.
3. **Phase 3 — Payload Metadata Inspection:** Analyzed HTTP `POST` multipart headers (`Content-Disposition`) and response bodies to verify the transmitted filename and confirm the destination directory on the target server.

4. **Phase 4 — Stream Timing & Latency Analysis:** Evaluated multiple timing boundaries within the upload TCP stream, contrasting HTTP application response latency (`http.time`), payload-to-response delta, and full transport-layer session duration (`SYN` handshake to `FIN/ACK` teardown) via `frame.time_relative` and `awk` .

4. Key Evidence & Technical Artifacts

OSI Layer / Domain	Observed Forensic Artifact	Investigative Status
Network Layer (IPv4)	Distinct sender/receiver IPv4 pairs separating workstation P13 traffic from the target HTTP server endpoint.	Tasks 1 & 2 — Task Completed
Application Layer (HTTP)	Cleartext HTTP/1.1 `POST` request to a PHP upload script containing multipart file metadata, paired with a `200 OK` response disclosing the web server software and destination upload folder.	Tasks 3, 4 & 5 — Task Completed
Transport Layer (TCP)	Complete TCP stream lifecycle comprising the 3-way handshake (`SYN` , `SYN-ACK` , `ACK`), segmented `PSH/ACK` data frames (`Len=1460`), server acknowledgments, and graceful connection closure (`FIN, ACK`).	Task 6 — Task Completed

> **Analytical Note on Timing Precision:** During Task 6, comparing application-layer response time (`http.time`) against transport-layer session duration demonstrated that full file transmission measurement requires computing the exact delta between the initial `SYN` frame and final `FIN/ACK` frame via `frame.time_relative` , formatted to four decimal places.

5. Lessons Learned & Hardening Recommendations

Analytical Takeaways

- **Metric Boundary Clarity:** In network forensics, transfer duration differs significantly depending on whether an analyst measures HTTP response latency or the full TCP stream lifecycle. Always verify transport-layer boundaries when quantifying data transfers.
- **CLI Triage Efficiency:** Combining `tshark` field extraction with `awk` arithmetic provides fast, reproducible timestamp calculations directly in the terminal without relying solely on GUI conversation statistics.

Defensive & Hardening Recommendations

- **Enforce TLS Encryption (HTTPS):** The file upload occurred over unencrypted HTTP (port 80), exposing filenames, upload endpoints, and server software banners in cleartext. Enforce TLS 1.2/1.3 across all web services.
- **Harden File Upload Handlers:** Implement strict server-side allowlisting for file extensions and MIME types on upload endpoints, randomize stored filenames, and disable script execution permissions inside upload directories.
- **Suppress Server Banners:** Configure web server directives (e.g., `ServerTokens Prod` and `ServerSignature Off`) to prevent disclosing software details in HTTP response headers.
- **Network Detection & Monitoring:** Deploy IDS/IPS signatures (Suricata/Snort) to detect cleartext HTTP `POST` multipart file transfers and alert on anomalous outbound workstation uploads.